

Security Implementation Analysis of ECC: TLS 1.3 Handshake

Joaquin Baltasar Villegas

Lander University

CIS-345-OL

Dr. Farha Ali

April 27, 2026

Introduction

Transport Layer Security (TLS) 1.3 is the protocol that protects much of today's Internet traffic, including web browsing, online banking, email, and cloud services. It provides confidentiality, integrity, and authentication so that sensitive data can move across an untrusted network without being read or altered by attackers (Internet Engineering Task Force, 2018). In practice, TLS 1.3 is the security layer that helps users safely log in, make payments, and exchange private information over HTTPS connections. Because this system protects passwords, financial records, and personal data, its security is critical.

Security is especially important in this context because the handshake is the point where the client and server agree on cryptographic settings and establish trust. If the handshake is weak or improperly implemented, an attacker may intercept traffic, impersonate a server, or downgrade the connection to weaker settings. TLS 1.3 was designed to reduce those risks by simplifying the handshake and removing older legacy mechanisms that had known weaknesses (Internet Engineering Task Force, 2018). For that reason, TLS 1.3 is a strong example of cryptography being used not just in theory, but as a practical defense for everyday communication.

Overview of the System

At a high level, TLS 1.3 works by having a client, such as a web browser, connect to a server, such as a banking website or cloud platform, and negotiate secure communication parameters. The client begins with a ClientHello message that proposes protocol versions, cipher suites, and supported key exchange methods. The server responds with ServerHello, selects the cryptographic parameters, and provides the information needed to continue the handshake securely (Internet Engineering Task Force, 2018). After the handshake is completed, both sides share derived session keys and use them to protect application data.

The system protects multiple kinds of data. This includes login credentials, bank account information, session tokens, private messages, API keys, and any other sensitive content transmitted over the network. TLS 1.3 also protects the integrity of the traffic, which means attackers cannot modify messages without being detected. The protocol therefore protects not only privacy, but also the authenticity and reliability of the communication channel (Internet Engineering Task Force, 2018).

Cryptography in Use

TLS 1.3 uses several cryptographic methods together rather than relying on one algorithm alone. For key exchange, it primarily uses Elliptic Curve Diffie-Hellman Ephemeral (ECDHE), which allows both sides to derive a shared secret without sending that secret over the network. This gives the connection forward secrecy, meaning compromise of a long-term private key does not automatically reveal past sessions (National Institute of Standards and Technology, 2018; Internet Engineering Task Force, 2018).

For authentication, TLS 1.3 typically uses digital signatures such as ECDSA or EdDSA. These signatures allow the server to prove its identity using a certificate chain trusted by the client. For key derivation, TLS 1.3 uses HKDF, which turns the shared secret into separate traffic keys for encryption and integrity protection. For data protection after the handshake, TLS 1.3 commonly

uses authenticated encryption algorithms such as AES-GCM or ChaCha20-Poly1305, which protect both confidentiality and tamper resistance at the same time (Internet Engineering Task Force, 2018). Hash functions are also used in the handshake transcript and key schedule to make sure the derived keys depend on the full exchange.

Real-World Application

A common example is a user logging into an online banking site. First, the browser sends a ClientHello message containing supported TLS versions and elliptic curve options. The bank's server responds with ServerHello and its certificate chain, which the browser checks against trusted certificate authorities. If the certificate is valid, the browser continues the handshake and accepts the server as authentic (Internet Engineering Task Force, 2018).

Next, the client and server each generate ephemeral elliptic curve key pairs and compute the same shared secret using ECDHE. That secret is never transmitted directly, which prevents passive attackers from learning it from network traffic. Both sides then apply HKDF to derive encryption keys and integrity keys for the session. Once the handshake is complete, all banking data, such as balances, transfers, and login tokens, is encrypted before transmission (Internet Engineering Task Force, 2018).

This step-by-step process is what makes TLS useful in practice. The cryptography is not just abstract math; it is the mechanism that allows the browser and server to trust each other and communicate securely across an unsafe network. Without these protections, a user's password or payment data could be exposed to interception or modification.

Security Analysis

The main strength of TLS 1.3 is that it improves security while simplifying the protocol. It removes legacy cipher suites, eliminates static RSA key exchange, and requires forward secrecy for standard public-key key exchange. It also encrypts handshake messages after ServerHello, which reduces the amount of metadata exposed to observers (Internet Engineering Task Force, 2018). These design choices make the protocol harder to attack than earlier versions.

Another strength is that elliptic curve cryptography offers strong security with relatively small keys and efficient computation. This makes it practical for high-volume systems such as web browsers, mobile apps, and cloud platforms. At the same time, TLS 1.3 is not risk-free. Security depends on correct implementation, and weaknesses can appear in key validation, randomness, side-channel resistance, or state-machine logic.

Known risks include invalid curve attacks, timing attacks, weak random number generation, and replay concerns with 0-RTT data. NIST guidance emphasizes proper public key validation in elliptic-curve key establishment, because failure to validate points can lead to serious compromise (National Institute of Standards and Technology, 2018). TLS 1.3 0-RTT also introduces security tradeoffs because early data may be replayed, so it is not suitable for every application (Cloudflare, 2017; Internet Society, 2018). In addition, future quantum computers could threaten current public-key systems, which is why U.S. guidance has begun shifting toward quantum-resistant algorithms (National Security Agency, 2022).

Conclusion

TLS 1.3 is a strong real-world example of how cryptography protects everyday digital communication. It uses elliptic curve key exchange, digital signatures, hash functions, and authenticated encryption to establish secure sessions and protect data in transit (Internet Engineering Task Force, 2018). Its streamlined handshake improves both security and performance compared with older TLS versions.

This application matters because modern life depends on secure communication for banking, messaging, identity verification, and cloud computing. At the same time, TLS 1.3 shows that cryptographic security depends on both the protocol design and the correctness of implementation. As security requirements evolve, TLS deployments will also need to adapt to new threats such as side channels and post-quantum cryptography migration (National Security Agency, 2022). In that sense, TLS 1.3 is both a current security foundation and a bridge to the future of secure communications.

References

- Cloudflare. (2017, March 14). *Introducing zero round trip time resumption (0-RTT)*.
<https://blog.cloudflare.com/introducing-0-rtt/09>
- Internet Engineering Task Force. (2018). *The transport layer security (TLS) protocol version 1.3 (RFC 8446)*. <https://datatracker.ietf.org/doc/html/rfc8446>
- Internet Society. (2018, August 25). *Deploying TLS 1.3*.
<https://www.internetsociety.org/blog/2018/08/deploying-tls-1-3/>
- National Institute of Standards and Technology. (2018, April). *Recommendation for pair-wise key-establishment schemes using discrete logarithm cryptography (NIST SP 800-56A Rev. 3)*. <https://doi.org/10.6028/NIST.SP.800-56Ar3>
- National Security Agency. (2022, September). *Announcing the Commercial National Security Algorithm Suite 2.0* [Cybersecurity advisory].
https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF